

COMS30035, Machine learning: Linear Regression, Linear Discriminant and Logistic Regression

James Cussens

School of Computer Science
University of Bristol

7th September 2026

Acknowledgement

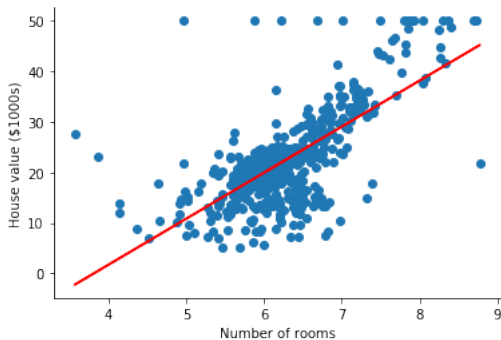
- ▶ These slides are adapted from ones originally created by [Rui Ponte Costa](#) and Dima Damen and later edited by Edwin Simpson.

Agenda

- ▶ Linear regression
- ▶ Nonlinear regression
- ▶ Probabilistic models
- ▶ Maximum likelihood estimation
- ▶ Discriminant functions
- ▶ Logistic regression

Revisiting regression

- ▶ Goal: Finding a relationship between two variables (e.g. regress *house value* against *number of rooms*)
- ▶ Model: Linear relationship between *house value* and *number of rooms*?



Revisiting regression – deterministic model

Data: a set of data points $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ where x_n is the number of rooms of house n and y_n the house value.

Task: build a model that can predict the house value from the number of rooms

Model Type: parametric; assumes a polynomial relationship between house value and number of rooms

Model Complexity: assume the relationship is linear
house value = $a_0 + a_1 \times \text{rooms}$

$$y_n = a_0 + a_1 x_n \quad (1)$$

Model Parameters: model has two parameters a_0 and a_1 which should be estimated.

- ▶ a_0 is the y-intercept
- ▶ a_1 is the slope of the line

Revisiting regression – deterministic model

Data: a set of data points $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ where x_n is the number of rooms of house n and y_n the house value.

Task: build a model that can predict the house value from the number of rooms

Model Type: parametric; assumes a polynomial relationship between house value and number of rooms

Model Complexity: assume the relationship is linear
house value = $a_0 + a_1 \times \text{rooms}$

$$y_n = a_0 + a_1 x_n \quad (1)$$

Model Parameters: model has two parameters a_0 and a_1 which should be estimated.

- ▶ a_0 is the y-intercept
- ▶ a_1 is the slope of the line

Revisiting regression – deterministic model

Data: a set of data points $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ where x_n is the number of rooms of house n and y_n the house value.

Task: build a model that can predict the house value from the number of rooms

Model Type: parametric; assumes a polynomial relationship between house value and number of rooms

Model Complexity: assume the relationship is linear
house value = $a_0 + a_1 \times \text{rooms}$

$$y_n = a_0 + a_1 x_n \quad (1)$$

Model Parameters: model has two parameters a_0 and a_1 which should be estimated.

- ▶ a_0 is the y-intercept
- ▶ a_1 is the slope of the line

Revisiting regression – deterministic model

Data: a set of data points $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ where x_n is the number of rooms of house n and y_n the house value.

Task: build a model that can predict the house value from the number of rooms

Model Type: parametric; assumes a polynomial relationship between house value and number of rooms

Model Complexity: assume the relationship is linear
house value = $a_0 + a_1 \times \text{rooms}$

$$y_n = a_0 + a_1 x_n \quad (1)$$

Model Parameters: model has two parameters a_0 and a_1 which should be estimated.

- ▶ a_0 is the y-intercept
- ▶ a_1 is the slope of the line

Revisiting linear regression – fitting

- ▶ Although we are currently assuming the model is linear, the data will not typically be consistent with such an assumption. So ...
- ▶ Find $\theta = (a_0, a_1)$ which minimises

$$R(a_0, a_1) = \sum_{n=1}^N (y_n - (a_0 + a_1 x_n))^2 \quad (2)$$

- ▶ R is an abbreviation for **residual sum of squares** (sometimes written RSS).
- ▶ Using matrix notation, then for any parameter vector θ , which defines a linear function, we have:

$$\frac{\partial R}{\partial \theta} = -2\mathbf{X}^T(\mathbf{y} - \mathbf{X}\theta)$$

- ▶ Here $X_{n,j}$ (the value at row n , column j of $\mathbf{X}$) is the value of the j th feature in the n th datapoint (where we add a fake feature which is always 1 to handle the intercept), and y_n is the value of the *response* in the n th datapoint.
- ▶ Setting $\frac{\partial R}{\partial \theta}$ to 0 and re-arranging we get:

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Least Squares Solution

Example

Find the best least squares fit by a linear function to the data using $p = 1$

x	-1	0	1	2
y	0	1	3	9

$$\mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$y = 1.8 + 2.9x$$

Least Squares Solution

Example

Find the best least squares fit by a linear function to the data using $p = 1$

x	-1	0	1	2
y	0	1	3	9

$$\mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$y = 1.8 + 2.9x$$

Least Squares Solution

Example

Find the best least squares fit by a linear function to the data using $p = 1$

x	-1	0	1	2
y	0	1	3	9

$$\mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$y = 1.8 + 2.9x$$

Least Squares Solution

Example

Find the best least squares fit by a linear function to the data using $p = 1$

x	-1	0	1	2
y	0	1	3	9

$$\mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$y = 1.8 + 2.9x$$

Least Squares Solution

Example

Find the best least squares fit by a linear function to the data using $p = 1$

x	-1	0	1	2
y	0	1	3	9

$$\mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$y = 1.8 + 2.9x$$

Least Squares Solution

Example

Find the best least squares fit by a linear function to the data using $p = 1$

x	-1	0	1	2
y	0	1	3	9

$$\mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$y = 1.8 + 2.9x$$

Least Squares Solution

Example

Find the best least squares fit by a linear function to the data using $p = 1$

x	-1	0	1	2
y	0	1	3	9

$$\mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$y = 1.8 + 2.9x$$

Least Squares Solution

Example

Find the best least squares fit by a linear function to the data using $p = 1$

x	-1	0	1	2
y	0	1	3	9

$$\mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$y = 1.8 + 2.9x$$

Least Squares Solution

Example

Find the best least squares fit by a linear function to the data using $p = 1$

x	-1	0	1	2
y	0	1	3	9

$$\mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$y = 1.8 + 2.9x$$

Least Squares Solution

Example

Find the best least squares fit by a linear function to the data using $p = 1$

x	-1	0	1	2
y	0	1	3	9

$$\mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$y = 1.8 + 2.9x$$

Computational methods for linear regression

- ▶ Although $\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$, $\hat{\theta}$ is never (well, should never) be computed by computing $(\mathbf{X}^T \mathbf{X})^{-1}$
- ▶ Instead *Singular Value Decomposition* or *QR decomposition* is used to derive $\hat{\theta}$.
- ▶ See [Mur22, §11.2.2.3] for further details.

Probabilistic model assumed when doing linear regression

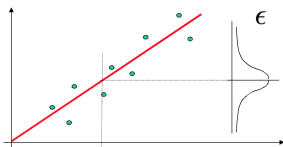
Probabilistic models are a core part of ML, as they represent uncertainty about future predictions, which is critical for real world applications. For simplicity, let's drop a_0 from the previous model. We add a random variable ϵ that captures the uncertainty of predicted house prices

$$\text{house price} = a_1 \times \text{number of rooms} + \epsilon$$

For linear regression, we assume that ϵ is given by $\mathcal{N}(\mu = 0, \sigma^2)$ —a zero mean Gaussian distribution—which gives the (conditional) **likelihood**

$$p(\mathbf{y}|\mathbf{X}, a_1, \sigma^2) = \prod_{n=1}^N p(\text{price}_n | \text{rooms}_n, a_1, \sigma^2) = \prod_{n=1}^N \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2} \frac{(\text{price}_n - a_1 \text{rooms}_n)^2}{\sigma^2}}$$

This distribution over $\mathbf{y}$ conditional on $\mathbf{X}$ is determined by the slope a_1 and variance σ^2 ¹



¹ Note that here $\mu = a_0$ which, for simplicity, we assume to be zero.

Probabilistic model assumed when doing linear regression

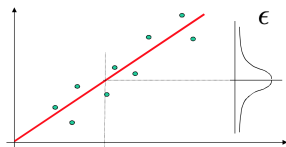
Probabilistic models are a core part of ML, as they represent uncertainty about future predictions, which is critical for real world applications. For simplicity, let's drop a_0 from the previous model. We add a random variable ϵ that captures the uncertainty of predicted house prices

$$\text{house price} = a_1 \times \text{number of rooms} + \epsilon$$

For linear regression, we assume that ϵ is given by $\mathcal{N}(\mu = 0, \sigma^2)$ —a zero mean Gaussian distribution—which gives the (conditional) **likelihood**

$$p(\mathbf{y}|\mathbf{X}, a_1, \sigma^2) = \prod_{n=1}^N p(\text{price}_n | \text{rooms}_n, a_1, \sigma^2) = \prod_{n=1}^N \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2} \frac{(\text{price}_n - a_1 \text{rooms}_n)^2}{\sigma^2}}$$

This distribution over $\mathbf{y}$ conditional on $\mathbf{X}$ is determined by the slope a_1 and variance σ^2 ¹



¹ Note that here $\mu = a_0$ which, for simplicity, we assume to be zero.

Probabilistic model assumed when doing linear regression

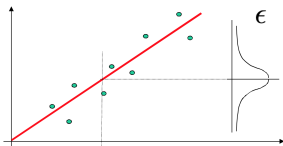
Probabilistic models are a core part of ML, as they represent uncertainty about future predictions, which is critical for real world applications. For simplicity, let's drop a_0 from the previous model. We add a random variable ϵ that captures the uncertainty of predicted house prices

$$\text{house price} = a_1 \times \text{number of rooms} + \epsilon$$

For linear regression, we assume that ϵ is given by $\mathcal{N}(\mu = 0, \sigma^2)$ —a zero mean Gaussian distribution—which gives the (conditional) **likelihood**

$$p(\mathbf{y}|\mathbf{X}, a_1, \sigma^2) = \prod_{n=1}^N p(\text{price}_n | \text{rooms}_n, a_1, \sigma^2) = \prod_{n=1}^N \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2} \frac{(\text{price}_n - a_1 \text{rooms}_n)^2}{\sigma^2}}$$

This distribution over $\mathbf{y}$ conditional on $\mathbf{X}$ is determined by the slope a_1 and variance σ^2 ¹



¹Note that here $\mu = a_0$ which, for simplicity, we assume to be zero.

Where's the estimate for σ^2 ?

- ▶ When we computed the least squares fit for some data back on slide 7, no estimate for σ^2 was given.
- ▶ If we want $\hat{\sigma}^2$, an estimate for σ^2 , it's easy enough to get from the observed errors:

$$\hat{\sigma}^2 = \frac{1}{N - p - 1} R(\hat{\theta}) = \frac{1}{N - p - 1} \sum_{n=1}^N (y_n - \hat{y}_n)^2$$

- ▶ N is the number of datapoints, p the number of estimated parameters and $\hat{y}_n$ is the predicted value for y_n .
- ▶ We divide by $N - p - 1$, rather than the more intuitive N , to get an **unbiased** estimator.
- ▶ Idea is: if N were small, but p were big, then $R(\hat{\theta})/N$ would underestimate the uncertainty. $R(\hat{\theta})$ might even be 0.

Maximum Likelihood Estimation

- ▶ **Maximum-likelihood estimation (MLE)** is a method of estimating the parameters of a probabilistic model.
- ▶ Assume θ is a vector of all parameters of the probabilistic model. (e.g. $\theta = \{a_1, \sigma^2\}$).
- ▶ MLE produces an estimate $\hat{\theta}$ of θ by maximising an **objective function** of θ .

Maximum Likelihood Estimation

- ▶ **Maximum-likelihood estimation (MLE)** is a method of estimating the parameters of a probabilistic model.
- ▶ Assume θ is a vector of all parameters of the probabilistic model. (e.g. $\theta = \{a_1, \sigma^2\}$).
- ▶ MLE produces an estimate $\hat{\theta}$ of θ by maximising an **objective function** of θ .

Maximising Objective Functions

Definition

- ▶ Assume $f(\theta)$ is an objective function to be optimised (e.g. maximised), the *arg max* value is that value of θ which gives the maximum value of the objective function f :

If

$$\hat{\theta} = \arg \max_{\theta} f(\theta)$$

then

$$\forall \theta : f(\hat{\theta}) \geq f(\theta)$$

- ▶ Here, we are assuming that there is only one $\hat{\theta}$ which maximises f . Of course, this is not always the case.
- ▶ 'Learning' the parameter is then equal to finding the maximum argument *arg max*.

Maximising Objective Functions

Definition

- ▶ Assume $f(\theta)$ is an objective function to be optimised (e.g. maximised), the *arg max* value is that value of θ which gives the maximum value of the objective function f :

If

$$\hat{\theta} = \arg \max_{\theta} f(\theta)$$

then

$$\forall \theta : f(\hat{\theta}) \geq f(\theta)$$

- ▶ Here, we are assuming that there is only one $\hat{\theta}$ which maximises f . Of course, this is not always the case.
- ▶ 'Learning' the parameter is then equal to finding the maximum argument *arg max*.

Maximising Objective Functions

Definition

- ▶ Assume $f(\theta)$ is an objective function to be optimised (e.g. maximised), the *arg max* value is that value of θ which gives the maximum value of the objective function f :

If

$$\hat{\theta} = \arg \max_{\theta} f(\theta)$$

then

$$\forall \theta : f(\hat{\theta}) \geq f(\theta)$$

- ▶ Here, we are assuming that there is only one $\hat{\theta}$ which maximises f . Of course, this is not always the case.
- ▶ ‘Learning’ the parameter is then equal to finding the maximum argument *arg max*.

Maximum Likelihood Estimation - General

- If D is the data, then the **likelihood** is simply $f(\theta) = P(D|\theta)$.

$$\begin{aligned}\theta_{MLE} &= \arg \max_{\theta} p(D|\theta) \\ &= \arg \max_{\theta} \ln p(D|\theta) \\ &= \arg \min_{\theta} -\ln p(D|\theta)\end{aligned}$$

Sometimes this recipe for MLE works:

1. Determine θ , D and expression for likelihood $p(D|\theta)$
2. Take the natural logarithm of the likelihood
3. Take the derivative of $\ln p(D|\theta)$ w.r.t. θ . If θ is a multi-dimensional vector, take partial derivatives
4. Set derivative(s) to 0 and solve for θ

Maximum Likelihood Estimation - General

- If D is the data, then the **likelihood** is simply $f(\theta) = P(D|\theta)$.

$$\begin{aligned}\theta_{MLE} &= \arg \max_{\theta} p(D|\theta) \\ &= \arg \max_{\theta} \ln p(D|\theta) \\ &= \arg \min_{\theta} -\ln p(D|\theta)\end{aligned}$$

Sometimes this recipe for MLE works:

1. Determine θ , D and expression for likelihood $p(D|\theta)$
2. Take the natural logarithm of the likelihood
3. Take the derivative of $\ln p(D|\theta)$ w.r.t. θ . If θ is a multi-dimensional vector, take partial derivatives
4. Set derivative(s) to 0 and solve for θ

Maximum Likelihood Estimation - General

- If D is the data, then the **likelihood** is simply $f(\theta) = P(D|\theta)$.

$$\begin{aligned}\theta_{MLE} &= \arg \max_{\theta} p(D|\theta) \\ &= \arg \max_{\theta} \ln p(D|\theta) \\ &= \arg \min_{\theta} -\ln p(D|\theta)\end{aligned}$$

Sometimes this recipe for MLE works:

1. Determine θ , D and expression for likelihood $p(D|\theta)$
2. Take the natural logarithm of the likelihood
3. Take the derivative of $\ln p(D|\theta)$ w.r.t. θ . If θ is a multi-dimensional vector, take partial derivatives
4. Set derivative(s) to 0 and solve for θ

Maximum Likelihood Estimation - General

- If D is the data, then the **likelihood** is simply $f(\theta) = P(D|\theta)$.

$$\begin{aligned}\theta_{MLE} &= \arg \max_{\theta} p(D|\theta) \\ &= \arg \max_{\theta} \ln p(D|\theta) \\ &= \arg \min_{\theta} -\ln p(D|\theta)\end{aligned}$$

Sometimes this recipe for MLE works:

1. Determine θ , D and expression for likelihood $p(D|\theta)$
2. Take the natural logarithm of the likelihood
3. Take the derivative of $\ln p(D|\theta)$ w.r.t. θ . If θ is a multi-dimensional vector, take partial derivatives
4. Set derivative(s) to 0 and solve for θ

Maximum Likelihood Estimation - General

- If D is the data, then the **likelihood** is simply $f(\theta) = P(D|\theta)$.

$$\begin{aligned}\theta_{MLE} &= \arg \max_{\theta} p(D|\theta) \\ &= \arg \max_{\theta} \ln p(D|\theta) \\ &= \arg \min_{\theta} -\ln p(D|\theta)\end{aligned}$$

Sometimes this recipe for MLE works:

1. Determine θ , D and expression for likelihood $p(D|\theta)$
2. Take the natural logarithm of the likelihood
3. Take the derivative of $\ln p(D|\theta)$ w.r.t. θ . If θ is a multi-dimensional vector, take partial derivatives
4. Set derivative(s) to 0 and solve for θ

Maximum Likelihood Estimation - General

- If D is the data, then the **likelihood** is simply $f(\theta) = P(D|\theta)$.

$$\begin{aligned}\theta_{MLE} &= \arg \max_{\theta} p(D|\theta) \\ &= \arg \max_{\theta} \ln p(D|\theta) \\ &= \arg \min_{\theta} -\ln p(D|\theta)\end{aligned}$$

Sometimes this recipe for MLE works:

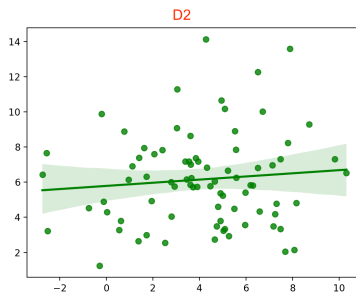
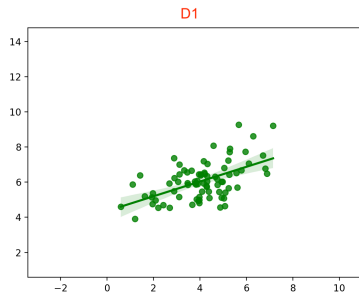
1. Determine θ , D and expression for likelihood $p(D|\theta)$
2. Take the natural logarithm of the likelihood
3. Take the derivative of $\ln p(D|\theta)$ w.r.t. θ . If θ is a multi-dimensional vector, take partial derivatives
4. Set derivative(s) to 0 and solve for θ

Least Squares and MLE for Linear Regression

- ▶ In the case of standard linear regression one can prove that the parameters which minimise the squared error are also the MLE parameters.
- ▶ Note that in general the MLE recipe on the previous slide will only find local maxima of the likelihood (so not necessarily the MLE).
- ▶ But in the special case of linear regression it does find the MLE.

Representing uncertainty

- ▶ If we apply MLE to two datasets (D_1 and D_2) what would the parameters $\theta = \{a_1, \sigma^2\}$ be?



- ▶ $\hat{a}_1(D_1) > \hat{a}_1(D_2)$ [slope] and $\hat{\sigma}^2(D_1) < \hat{\sigma}^2(D_2)$ [uncertainty]
- ▶ Note that the linear regression model assumes the level of uncertainty is constant, but in both these data plots this appears not to be the case.
- ▶ So linear regression is perhaps not the best model for this data.

Classification

- ▶ Classification is an example of **supervised learning**
- ▶ Goal: Classify input data into one of K classes
- ▶ Model: *Discriminant function*:
 - ▶ A function that takes an input vector x and assigns it to class C_k . For simplicity we will focus on $K = 2$ and will first study linear functions (see Bishop for the general cases).

Classification

- ▶ Classification is an example of **supervised learning**
- ▶ Goal: Classify input data into one of K classes
- ▶ Model: *Discriminant function*:
 - ▶ A function that takes an input vector x and assigns it to class C_k . For simplicity we will focus on $K = 2$ and will first study linear functions (see Bishop for the general cases).

Linear discriminant function

- ▶ The simplest linear discriminant (LD) is $y(\mathbf{x}) = w_0 + \mathbf{w}^T \mathbf{x}$
 - ▶ where y is used to predict class C_k , $\mathbf{x}$ is the input vector (feature values)
 - ▶ w_0 is a scalar, which we call *bias*
 - ▶ $\mathbf{w}$ is our vector of parameters, which we call *weights*
- ▶ This looks like linear regression! Except the next step...
- ▶ For $K = 2$: An input vector $\mathbf{x}$ is assigned to class C_1 if $y(\mathbf{x}) \geq 0$ and to class C_2 otherwise.
- ▶ A direct application of least-squares to choose w_0 and $\mathbf{w}$ does not give great results. See Bishop §4.1.3
- ▶ Instead we can assume the data from each class has a multivariate Gaussian distribution whose mean is class-specific (but where the covariance matrix for each class is the same), use MLE to find parameters for each of these Gaussians and finally use Bayes theorem to assign classes. See [scikit-learn explanation](#).

Linear discriminant function

- ▶ The simplest linear discriminant (LD) is $y(\mathbf{x}) = w_0 + \mathbf{w}^T \mathbf{x}$
 - ▶ where y is used to predict class C_k , $\mathbf{x}$ is the input vector (feature values)
 - ▶ w_0 is a scalar, which we call *bias*
 - ▶ $\mathbf{w}$ is our vector of parameters, which we call *weights*
- ▶ This looks like linear regression! Except the next step...
- ▶ For $K = 2$: An input vector $\mathbf{x}$ is assigned to class C_1 if $y(\mathbf{x}) \geq 0$ and to class C_2 otherwise.
- ▶ A direct application of least-squares to choose w_0 and $\mathbf{w}$ does not give great results. See Bishop §4.1.3
- ▶ Instead we can assume the data from each class has a multivariate Gaussian distribution whose mean is class-specific (but where the covariance matrix for each class is the same), use MLE to find parameters for each of these Gaussians and finally use Bayes theorem to assign classes. See [scikit-learn explanation](#).

Linear discriminant function

- ▶ The simplest linear discriminant (LD) is $y(\mathbf{x}) = w_0 + \mathbf{w}^T \mathbf{x}$
 - ▶ where y is used to predict class C_k , $\mathbf{x}$ is the input vector (feature values)
 - ▶ w_0 is a scalar, which we call *bias*
 - ▶ $\mathbf{w}$ is our vector of parameters, which we call *weights*
- ▶ This looks like linear regression! Except the next step...
- ▶ For $K = 2$: An input vector $\mathbf{x}$ is assigned to class C_1 if $y(\mathbf{x}) \geq 0$ and to class C_2 otherwise.
- ▶ A direct application of least-squares to choose w_0 and $\mathbf{w}$ does not give great results. See Bishop §4.1.3
- ▶ Instead we can assume the data from each class has a multivariate Gaussian distribution whose mean is class-specific (but where the covariance matrix for each class is the same), use MLE to find parameters for each of these Gaussians and finally use Bayes theorem to assign classes. See [scikit-learn explanation](#).

Linear discriminant function

- ▶ The simplest linear discriminant (LD) is $y(\mathbf{x}) = w_0 + \mathbf{w}^T \mathbf{x}$
 - ▶ where y is used to predict class C_k , $\mathbf{x}$ is the input vector (feature values)
 - ▶ w_0 is a scalar, which we call *bias*
 - ▶ $\mathbf{w}$ is our vector of parameters, which we call *weights*
- ▶ This looks like linear regression! Except the next step...
- ▶ For $K = 2$: An input vector $\mathbf{x}$ is assigned to class C_1 if $y(\mathbf{x}) \geq 0$ and to class C_2 otherwise.
- ▶ A direct application of least-squares to choose w_0 and $\mathbf{w}$ does not give great results. See Bishop §4.1.3
- ▶ Instead we can assume the data from each class has a multivariate Gaussian distribution whose mean is class-specific (but where the covariance matrix for each class is the same), use MLE to find parameters for each of these Gaussians and finally use Bayes theorem to assign classes. See [scikit-learn explanation](#).

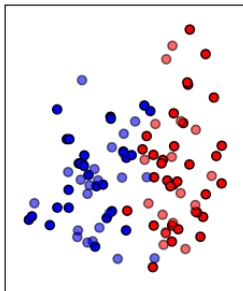
Linear discriminant function

- ▶ The simplest linear discriminant (LD) is $y(\mathbf{x}) = w_0 + \mathbf{w}^T \mathbf{x}$
 - ▶ where y is used to predict class C_k , $\mathbf{x}$ is the input vector (feature values)
 - ▶ w_0 is a scalar, which we call *bias*
 - ▶ $\mathbf{w}$ is our vector of parameters, which we call *weights*
- ▶ This looks like linear regression! Except the next step...
- ▶ For $K = 2$: An input vector $\mathbf{x}$ is assigned to class C_1 if $y(\mathbf{x}) \geq 0$ and to class C_2 otherwise.
- ▶ A direct application of least-squares to choose w_0 and $\mathbf{w}$ does not give great results. See Bishop §4.1.3
- ▶ Instead we can assume the data from each class has a multivariate Gaussian distribution whose mean is class-specific (but where the covariance matrix for each class is the same), use MLE to find parameters for each of these Gaussians and finally use Bayes theorem to assign classes. See [scikit-learn explanation](#).

LD and linear separability

- ▶ “Data sets whose classes can be separated exactly by linear decision surfaces are said to be *linearly separable*.” [Bis06, p.179].
- ▶ We generated two sets of points using two Gaussians to illustrate this point, which can easily be fit by a LD.
- ▶ A *decision boundary* is the boundary that separates the two given classes—on one side red is more likely, on the other blue—which our models will try to find.
- ▶ Note that in the data we can have a red dot where a blue is more likely, just by chance.

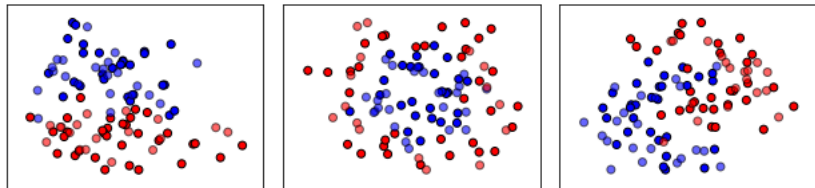
Example



Linear separability vs nonlinear separability

Example

Which datasets have classes which **are** and **are not** linearly separable²?



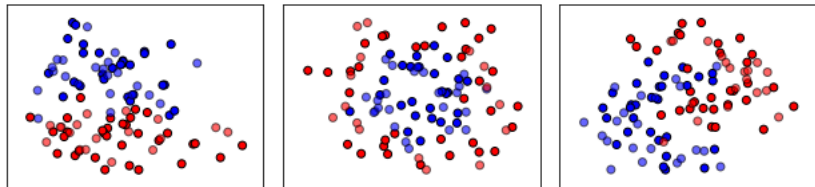
(Looks like) only the first dataset has linearly separable classes.

²Example from Sklearn [here](#).

Linear separability vs nonlinear separability

Example

Which datasets have classes which **are** and **are not** linearly separable²?



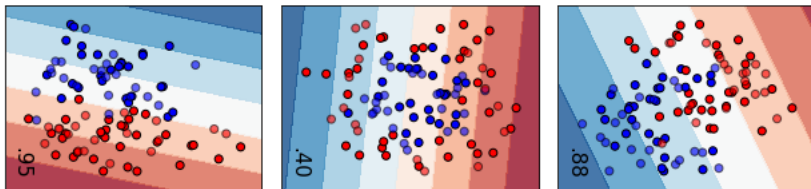
(Looks like) only the first dataset has linearly separable classes.

²Example from Sklearn [here](#).

Linear discriminant

Example

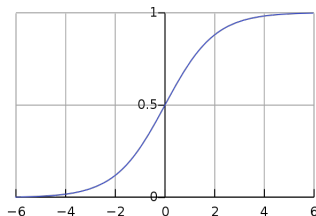
Using sklearn we fitted a LD to the data:



As expected, the LD model only does a good job in finding a good separation in the first dataset.

Logistic regression

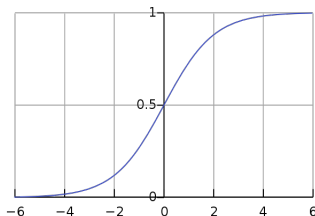
- Assume again we have only two classes. We use a logistic function to obtain the probability of class C_k : $y(\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x})$ where σ denotes the logistic sigmoid function (s-shaped), for example:



- $\sigma(\eta) = 1/(1 + \exp(-\eta))$
- Taking a probabilistic view:
 $p(C_1|\mathbf{x}) = y(\mathbf{x})$, and $p(C_2|\mathbf{x}) = 1 - p(C_1|\mathbf{x})$.
- NB. σ does not represent standard deviation here!

Logistic regression

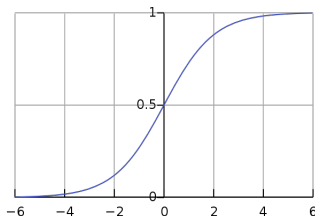
- Assume again we have only two classes. We use a logistic function to obtain the probability of class C_k : $y(\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x})$ where σ denotes the logistic sigmoid function (s-shaped), for example:



- $\sigma(\eta) = 1/(1 + \exp(-\eta))$
- Taking a probabilistic view:
 $p(C_1|\mathbf{x}) = y(\mathbf{x})$, and $p(C_2|\mathbf{x}) = 1 - p(C_1|\mathbf{x})$.
- NB. σ does not represent standard deviation here!

Logistic regression

- Assume again we have only two classes. We use a logistic function to obtain the probability of class C_k : $y(\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x})$ where σ denotes the logistic sigmoid function (s-shaped), for example:



- $\sigma(\eta) = 1/(1 + \exp(-\eta))$
- Taking a probabilistic view:
 $p(C_1|\mathbf{x}) = y(\mathbf{x})$, and $p(C_2|\mathbf{x}) = 1 - p(C_1|\mathbf{x})$.
- NB. σ does not represent standard deviation here!

Logistic regression – maximum likelihood estimation

Follow MLE recipe:

1. Define likelihood: For a dataset $\{\mathbf{x}_n, t_n\}$, where the targets

$$t_n \in \{0, 1\} \text{ we have } p(\mathbf{t}|\mathbf{x}, \mathbf{w}) = \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n} \text{ where}$$

$$y_n = p(C_1|\mathbf{x}_n).^3$$

2. Take negative logarithm of the likelihood:

$$-\ln p(\mathbf{t}|\mathbf{x}, \mathbf{w}) = -\sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}$$

3. Calculate the derivative w.r.t. the parameters $\mathbf{w}$:⁴

$$\frac{d \ln p(\mathbf{t}|\mathbf{x}, \mathbf{w})}{d\mathbf{w}} = \sum_{n=1}^N (y_n - t_n) \mathbf{x}_n$$

4. Use this derivative (gradient) to iteratively improve $\mathbf{w}$.

³The exponent selects the probability of the target class (i.e. if $t_n = 1$ we get y_n ; if $t_n = 0$ we get $1 - y_n$).

⁴This solution makes sense since we want to optimise the difference between the model output y and the desired targets t .

Logistic regression – maximum likelihood estimation

Follow MLE recipe:

1. Define likelihood: For a dataset $\{\mathbf{x}_n, t_n\}$, where the targets

$$t_n \in \{0, 1\} \text{ we have } p(\mathbf{t}|\mathbf{x}, \mathbf{w}) = \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n} \text{ where}$$

$$y_n = p(C_1|\mathbf{x}_n).^3$$

2. Take negative logarithm of the likelihood:

$$-\ln p(\mathbf{t}|\mathbf{x}, \mathbf{w}) = -\sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}$$

3. Calculate the derivative w.r.t. the parameters $\mathbf{w}$:⁴

$$\frac{d \ln p(\mathbf{t}|\mathbf{x}, \mathbf{w})}{d\mathbf{w}} = \sum_{n=1}^N (y_n - t_n) \mathbf{x}_n$$

4. Use this derivative (gradient) to iteratively improve $\mathbf{w}$.

³The exponent selects the probability of the target class (i.e. if $t_n = 1$ we get y_n ; if $t_n = 0$ we get $1 - y_n$).

⁴This solution makes sense since we want to optimise the difference between the model output y and the desired targets t .

Logistic regression – maximum likelihood estimation

Follow MLE recipe:

1. Define likelihood: For a dataset $\{x_n, t_n\}$, where the targets

$$t_n \in \{0, 1\} \text{ we have } p(\mathbf{t}|\mathbf{x}, \mathbf{w}) = \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n} \text{ where}$$

$$y_n = p(C_1|x_n).^3$$

2. Take negative logarithm of the likelihood:

$$-\ln p(\mathbf{t}|\mathbf{x}, \mathbf{w}) = -\sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}$$

3. Calculate the derivative w.r.t. the parameters $\mathbf{w}$:⁴

$$\frac{d \ln p(\mathbf{t}|\mathbf{x}, \mathbf{w})}{d\mathbf{w}} = \sum_{n=1}^N (y_n - t_n) x_n$$

4. Use this derivative (gradient) to iteratively improve $\mathbf{w}$.

³The exponent selects the probability of the target class (i.e. if $t_n = 1$ we get y_n ; if $t_n = 0$ we get $1 - y_n$).

⁴This solution makes sense since we want to optimise the difference between the model output y and the desired targets t .

Logistic regression – maximum likelihood estimation

Follow MLE recipe:

1. Define likelihood: For a dataset $\{x_n, t_n\}$, where the targets

$$t_n \in \{0, 1\} \text{ we have } p(\mathbf{t}|\mathbf{x}, \mathbf{w}) = \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n} \text{ where}$$

$$y_n = p(C_1|x_n).^3$$

2. Take negative logarithm of the likelihood:

$$-\ln p(\mathbf{t}|\mathbf{x}, \mathbf{w}) = -\sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}$$

3. Calculate the derivative w.r.t. the parameters $\mathbf{w}$:⁴

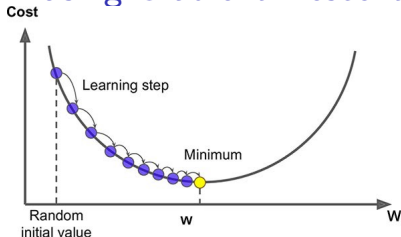
$$\frac{d \ln p(\mathbf{t}|\mathbf{x}, \mathbf{w})}{d\mathbf{w}} = \sum_{n=1}^N (y_n - t_n) x_n$$

4. Use this derivative (gradient) to iteratively improve $\mathbf{w}$.

³The exponent selects the probability of the target class (i.e. if $t_n = 1$ we get y_n ; if $t_n = 0$ we get $1 - y_n$).

⁴This solution makes sense since we want to optimise the difference between the model output y and the desired targets t .

MLE using Gradient Descent



- ▶ Start with random weight values
- ▶ We want to adjust each weight in $\mathbf{w}$ to minimise negative log likelihood: move downhill to the minimum
- ▶ The derivative represents the slope: $\frac{d \ln p(\mathbf{t}|\mathbf{x}, \mathbf{w})}{d\mathbf{w}} = \sum_{n=1}^N (y_n - t_n) \mathbf{x}_n$
- ▶ Increase or decrease $\mathbf{w}$ by a small amount in the downward direction
- ▶ In the particular case of logistic regression the error function is *convex* which means it has a unique minimum.
- ▶ NB. The 2006 edition of the Bishop book incorrectly states that the error function is *concave*. This was corrected in 2009.

Logistic regression – maximum likelihood estimation

More details on calculating the derivative:

1. From here $-\ln p(\mathbf{t}|\mathbf{x}, \mathbf{w}) = -\sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}$

2. We get $\sum_{n=1}^N \left\{ -\frac{t_n}{y_n} + \frac{(1-t_n)}{1-y_n} \right\} \{y_n(1-y_n)\} x_n$ ⁵

3. The above simplifies to $\sum_{n=1}^N \{-t_n(1-y_n) + (1-t_n)y_n\} x_n$

4. And in turn to $\sum_{n=1}^N \{y_n - t_n\} x_n$ ⁶

⁵We used the chain rule and $d \ln(x) = 1/x$. We also used the derivative of the sigmoid $dy_n = y(1 - y_n)$.

⁶You can find the full derivation [here](#).

Logistic regression – maximum likelihood estimation

More details on calculating the derivative:

1. From here $-\ln p(\mathbf{t}|\mathbf{x}, \mathbf{w}) = -\sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}$

2. We get $\sum_{n=1}^N \left\{ -\frac{t_n}{y_n} + \frac{(1-t_n)}{1-y_n} \right\} \{y_n(1-y_n)\} x_n$ ⁵

3. The above simplifies to $\sum_{n=1}^N \{-t_n(1-y_n) + (1-t_n)y_n\} x_n$

4. And in turn to $\sum_{n=1}^N \{y_n - t_n\} x_n$ ⁶

⁵We used the chain rule and $d \ln(x) = 1/x$. We also used the derivative of the sigmoid $dy_n = y(1 - y_n)$.

⁶You can find the full derivation [here](#).

Logistic regression – maximum likelihood estimation

More details on calculating the derivative:

1. From here $-\ln p(\mathbf{t}|\mathbf{x}, \mathbf{w}) = -\sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}$

2. We get $\sum_{n=1}^N \left\{ -\frac{t_n}{y_n} + \frac{(1-t_n)}{1-y_n} \right\} \{y_n(1-y_n)\} x_n$ ⁵

3. The above simplifies to $\sum_{n=1}^N \{-t_n(1-y_n) + (1-t_n)y_n\} x_n$

4. And in turn to $\sum_{n=1}^N \{y_n - t_n\} x_n$ ⁶

⁵We used the chain rule and $d \ln(x) = 1/x$. We also used the derivative of the sigmoid $dy_n = y(1 - y_n)$.

⁶You can find the full derivation [here](#).

Logistic regression – maximum likelihood estimation

More details on calculating the derivative:

1. From here $-\ln p(\mathbf{t}|\mathbf{x}, \mathbf{w}) = -\sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}$

2. We get $\sum_{n=1}^N \left\{ -\frac{t_n}{y_n} + \frac{(1-t_n)}{1-y_n} \right\} \{y_n(1-y_n)\} x_n$ ⁵

3. The above simplifies to $\sum_{n=1}^N \{-t_n(1-y_n) + (1-t_n)y_n\} x_n$

4. And in turn to $\sum_{n=1}^N \{y_n - t_n\} x_n$ ⁶

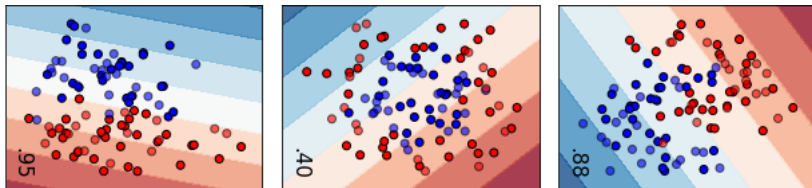
⁵We used the chain rule and $d \ln(x) = 1/x$. We also used the derivative of the sigmoid $dy_n = y(1 - y_n)$.

⁶You can find the full derivation [here](#).

Logistic regression

Example

Using sklearn we fitted a logistic regression classifier to the data:



As you can see the results are very similar to LD, but because of probabilistic formulation we have an explicit probability of belonging to one or the other class (not shown); this can be very useful in real-world applications (e.g. self-driving cars or cancer detection).

Optimisation algorithms for logistic regression

- ▶ Although we have looked at simple gradient descent for logistic regression,
- ▶ the traditional optimisation algorithm (*iterative reweighted least squares*) also uses the Hessian - the matrix of second partial derivatives.
- ▶ For logistic regression, scikit-learn offers you a choice of no fewer than 6 optimisation algorithms to choose from!

Reading

- ▶ Bishop §3.1 up to end of §3.1.1.
- ▶ Bishop Chapter 4 up to end of §4.1.1.
- ▶ Bishop §4.3.2
- ▶ Murphy §4.2.1–§4.2.2
- ▶ Murphy §10.1–§10.2.3.1
- ▶ Murphy §11.1–§11.2.2.1

Problems and quizzes

- ▶ No problems.
- ▶ Quizzes:
 - ▶ Week 1: Regression
 - ▶ Week 1: Classification



Christopher M. Bishop.

Pattern Recognition and Machine Learning.

Springer, 2006.



Kevin P. Murphy.

Probabilistic Machine Learning: An introduction.

MIT Press, 2022.